Tidal Pool: An Objective, Scalable, Semi-Persistent Bitcoin Transaction Pool

Eric Voskuil
eric@voskuil.org*

12 November 2026

Abstract

Libbitcoin version 4 archives an unconfirmed transaction exactly as it archives a confirmed one, and represents the transaction pool as an index over the archive rather than as a separate in-memory structure. This paper develops the admission rule and the data structures by which such a pool can hold *conflicting* unconfirmed transactions, so that replacement does not require a node to choose a winner, while still bounding the bytes a node will store against the fees that pay for them. The rule is a marginal coverage test: the fee a candidate adds to the best conflict-free outcome of its conflict component must pay for the candidate's own block space at the node's minimum rate. We show that this single-rate rule telescopes into the invariant that the best outcome of every component pays for every byte ever admitted into it, that it subsumes the usual own-rate and absolute-fee tests, and that it requires no incentive-compatibility check because no resolution is performed. Conflict components are recovered by label propagation over word-aligned columns of the tx pool table, with no stored graph, in a form that runs as a frontier walk on a CPU and as whole-column kernels on SIMD lanes or a GPU from the same template bodies. We describe the persistent label index, its maintenance across admission, confirmation, reorganization and prune, its relation to the archive's duplicate and point indexes, and the limits on members, outcomes and block fit that make the computation bounded. We compare the design with bitcoind's cluster mempool, from which it differs in three fundamental respects, and estimate admission cost for both at three levels of conflict complexity and for larger candidate sets.

1 Introduction

A Bitcoin node that relays unconfirmed transactions must decide which of them to keep. The decision is a denial-of-service question before it is an economic one: every kept transaction costs the node storage, bandwidth and validation, and the only payment on offer is a fee that the node itself will not collect unless it mines. The conventional answer, embodied in bitcoind's mempool, is to keep a conflict-free set, resolving conflicts at admission so that the set is directly mineable and relayable. Libbitcoin's architecture makes a different answer natural. Its pool is an index over archived transactions, so a kept transaction is already in the store, and the store is designed for vectorized scans over aligned columns. This paper develops the answer that follows: keep every conflicting transaction that pays for itself, and let the party with the mining incentive choose among them.

The tx pool is the middle of three aspects that work together. Relay decides what to announce to each peer, and the narrowcast design [8] settles that: packages are per-channel,

*Libbitcoin. This paper was drafted from the design sessions of 12 November 2026 with the assistance of Claude; the design, its history and all decisions recorded here are the author's. Cost figures are estimates from the stated models and are not measurements.

moment-in-time groupings built by nodes against the peer's advertised rate, never wallet artifacts, and an orphan is never received. Mining decides what to put in a block, and that belongs to the version 5 template graph. The pool decides what to keep, and this paper is about that decision. The three share one unit, a set that fits a block, and one accounting, fee against chargeable bytes; what this paper adds is the rule by which conflicting sets are kept rather than resolved, and the index that makes the rule cheap. Section 10 returns to how the same walk serves the other two aspects, and to fee estimation, which is a projection of the mining aspect.

Section 2 traces the design from the version 4 store. Section 3 sets out the model of components and outcomes. Section 4 develops the admission rule and proves its properties. Section 5 gives the label propagation algorithm and its two computational forms. Section 6 specifies the persistent index and its maintenance. Section 7 describes the computational pattern shared with the library's hash and curve code. Section 8 compares the design with bitcoind's cluster mempool. Section 9 estimates costs. Section 10 records open items.

2 History of the design

2.1 The 2021 node design

The design of the version 4 node [1] (version 8, 6 November 2021), published on the project site, set the premises from which the present work proceeds. Its optimization goals are stated in order: denial-of-service protection first, node restartability second, block template deduction speed third, then bootstrap speed, query consistency, query response time and storage efficiency. The pool design here is answerable to the first and third of these, in that order.

On double spending the document is explicit. "Transactions are not validated for double spending, as this is valid for a transaction (blocks impose order)." "Unconfirmed double spends are otherwise valid transactions." And in full: "Transactions are never declared valid as connectedness is sufficient for inclusion, with double-spending verified at the time of block confirmation. Double-spending is block-contextual and otherwise perfectly acceptable. Ordering transactions to resolve double spends is the reason that blocks exist. Double-spend is the property of a set of transactions, not any individual." Conflict coexistence in the pool is the direct consequence of that position; the present paper supplies the admission rule that makes it safe.

On the pool's guard: "Cost of confirmability is the pool DoS guard. Pool acceptance requires connection to other transactions that are either currently confirmed or connected in pool context. Double spending a confirmed transaction is also criteria for pool exclusion." Confirmed transactions as boundaries, and conflict with a confirmed spend as exclusion, are both already here. The document also anticipates a *transaction pool graph* that establishes confirmability, lists "transaction pool graph population" among the startup table scans, caches each transaction's fee "for population of the transaction graph" alongside its sigops and validation context, and restricts server query to the confirmed (by reverse index) and the confirmable (by pool graph). The graph was conceived as a structure to be populated; the present design derives it instead (Section 5).

Three further elements of the 2021 design recur below. Validation context is recorded per transaction, and "a previously validated transaction remains valid under the same validation context (forks, minimum time, and minimum height); if context differs then the transaction is revalidated", which is the basis of the context treatment in Section 4. Pool transactions are validated "in context of top block flags, median time past, and ++height", the pool context of the current chaser. And a hashmap index of pooled transactions by compact block short identifier is specified, populated from transactions that enter the pool, with short identifiers "computed from transaction hash". That element was wrong on two counts. BIP152 short identifiers are derived from the witness transaction hash, not the transaction hash, and they are keyed per block by a

nonce from the compact block header, so no static index by short identifier is possible. What is implemented is a `wtxid` index aligned with the pool, and four word-aligned witness-hash lane columns (`id0`..`id3`) over which each arriving compact block's SipHash correlation is run as a SIMD scan of the whole pool, or on a GPU should pools grow to warrant it. The scan columns of the tx pool table are the result of that correction, and the present design adds its columns beside them without disturbing the scan. The document's per-transaction checks distinguish consensus from DoS, with the DoS checks applied to pool transactions only "(block has PoW)", which is the reason compact block transactions are exempt from coverage.

2.2 The version 4 store

The companion relational database design [2] (10 November 2021) defined the store as a small set of memory-mapped primitives, array, blob, hashmap and multimap, with a strictly relational table design optimized for constant-time domain queries, compact storage, in-memory offset navigation and append-only writes. Two properties of that design bear directly on the present work.

First, the store has only two natural-key entry points, header hash and transaction hash, and only one place where a search is required to navigate the archive, output to spending inputs. That is the natural data model of the canonical serialization, an input names the output it spends and nothing names the input, preserved so that blocks archive in parallel and bodies stay append only, which is what makes failure recoverable by stopping.[1] The search is the point multimap: all spenders of a transaction's outputs share one stored instance of its hash, and the inputs that spend an output are found by searching that multimap by foreign point. It is performed once per validated input, where it is logically necessary, and never under milestone sync. The design document observes that double spenders are never obtained during validation, so no spender index is populated there. The duplicate table, which records points spent by more than one archived transaction, is the later resolution of this observation: it is written at archival when a point push collides with an existing spender, and its absence proves single spending without a search [5].[2]

Second, the document describes optional *mining optimization tables* that index previously-queried input-to-output and output-to-inputs relations, and lists among optional performance features "tx buffering to reduce RAM consumption during concurrent block validation" [2]. The pool of version 4 descends from this: it is a cache table in the store, not a structure in the node, and its rows exist to make block confirmation and compact-block fill cheap.

2.3 The node before a pool implementation

The libbitcoin-node wiki of 2024 documented the transaction chaser as a stub "awaiting a mempool design", and recorded the intended skeleton as "the obvious mempool: a partial DAG ordered by dependencies, with insertion emitting per-tx events" [3]. That reading of the 2021 graph as a stored dependency structure is what the present design replaces.

2.4 The pool as an index

The current architecture documentation [4, 6] fixes the properties on which this paper builds. The tx pool is an index over archived transactions: a submitted transaction is validated, archived

[1] Append-only bodies and the fail-stop recovery they permit are set out under store architecture on the project site.

[2] The store relies on eventual consistency, so two instances of one transaction can be archived: an invalid or reorganized block archived under proof of work during initial block download, the two coinbases of the BIP30 exception, or archival interrupted by a full disk. The table is a sink for those cases and is generally empty, and the collision test itself is almost always answered by the bloom filter kept in the point table's memory-fenced head cells, without touching the body.

like any other, and marked pooled under the context of the next block. There is no separate mempool structure; pooled is a store property, and the pool's contents outlive the process. The pool is opened by a one-way latch recorded in the store envelope when relay is enabled and the confirmed chain is current; transactions archived before the latch are never pooled. Every admission decision is made against the pool context, the chain context of the block after the confirmed top. Validation of a submitted package proceeds through consensus checks, prevout population from confirmed and pooled transactions, acceptance, the block-level guards a transaction would face in the next block, and script validation last, as the most costly step [4].

2.5 Version 4 and version 5

The version 4 pool is flat. It records, per pooled transaction, the context under which it was validated, its fee, size and sigops, its parent links and its time, but no graph object. A transaction metadata graph for block templates and acceptability is planned for the next major version, and the 2021 design anticipated both an object model for it and hardware acceleration of its processing for template generation. The work described here is the partial step that version 4 takes toward replacement: a query against the tx pool, for denial-of-service protection only, that permits conflicting transactions and packages to be admitted. Transactions arriving in compact blocks are exempt, since they are under proof of work.

What follows is an implementation of the designed behavior, not a change to it. The graph is still the object of the design; the question of how it would be processed on a GPU led to the observation that every operation on it is embarrassingly parallel over rows and inputs, and therefore that the logical graph can remain in the table, derived by passes over its columns rather than held as a structure. Nothing a stored graph would answer is answered differently.

3 Model

3.1 Rows, parents and boundaries

The tx pool table has one row per pooled transaction, keyed by the transaction's archive link. Let R be the set of rows. Each row r has a set of parents $P(r)$, the transactions whose outputs its inputs spend. A parent is either another pool row or a transaction confirmed in the chain.

Definition 3.1 (Boundary)**.** *A confirmed transaction is a* boundary*: it may be a parent of pool rows, but it is not a member of any component and no edge passes through it.*

The reason is that only a parent that can lose a conflict matters to ancestry. A confirmed parent cannot lose, so two of its children are not related through it. Without boundaries every descendant of one confirmed transaction would form one component. The pool terminates at confirmation: nothing is written to the pool at confirmation, and the rows of confirmed transactions, and of transactions made unconfirmable by a confirmation, are purged by prune.

3.2 Contests and cliques

Definition 3.2 (Contest and clique)**.** *A point (an output reference) is* contested *when two or more present pool rows spend it. The set of its spenders is the point's* clique*. At most one member of a clique can confirm.*

Contests are detected from the archive, not from the pool. The archive's point multimap yields every spender of a point, and the duplicate table is the set of all points with more than one spender [5]. The duplicate table arises in practice from duplicated transactions, since a transaction archived twice re-pushes every one of its points, but it records the same entry for a genuine conflict, and consensus reads it as the set of points that may have conflicting spenders.

It is therefore exactly the pool-wide list of contests. Duplicate instances of a transaction, which arise primarily when a disk-full condition during initial block download causes a block to be redownloaded, are a different matter and are handled at the archive level: the hash lookup settles on one instance and only that instance is ever pooled.

3.3 Components and the contested closure

Definition 3.3 (Contested closure)**.** *A row is* relevant *if it belongs to a clique or has a relevant parent. The relevant rows are the* contested closure*: clique members and all of their pool descendants.*

Definition 3.4 (Component)**.** *A* component *is a maximal set of relevant rows connected by parent–child edges and by clique membership.*

A row connected to a clique only as an uncontested ancestor, or as a descendant of an uncontested row, is not relevant. Its fee is the same in every outcome of the component and cancels in every comparison the admission rule makes. Consequently an uncontested chain of any length forms no component, and the member bound of Section 4.5 applies only to contested rows and their descendants. An earlier form of the design defined components over all parent–child edges; it would have refused a candidate joining a conflict-free cluster of more than N pooled transactions, which exchange batching chains and large fee-bumping trees reach easily.

3.4 Outcomes

Definition 3.5 (Outcome)**.** *An* outcome *of a component is a selection of one winner per clique, with the losers and all of their descendants removed. The remaining set is conflict-free and ancestry-closed by construction.*

The number of outcomes is the product of the clique sizes, and an outcome is decoded from its index by mixed radix over the cliques. Two selections may yield the same set when a winner descends from a loser; this is harmless to the admission rule, which takes maxima. The fee of an outcome is the sum of its members' fees.

4 The admission rule

4.1 Chargeable size

A block has one million virtual bytes and eighty thousand signature operations. Each sigop is therefore worth twelve and a half virtual bytes of block space, and bitcoind's policy rounds that to twenty. We price the larger of the two resources.

Definition 4.1 (Chargeable size)**.** *The chargeable size of a transaction t with virtual size $v(t)$ and sigop count $\sigma(t)$ is*

$$s(t) = \max\big(v(t),\ \beta\,\sigma(t)\big),$$

where β is the configured bytes per sigop.

bitcoind applies the same adjustment to every mempool entry, and its cluster mempool inherits it, so on this point the two designs coincide. The tx pool table already stores fee, virtual size and sigops per row, so pricing sigops changes what the rule receives and nothing in the tables or passes.

4.2 The marginal rule

Let C be the set of candidate rows of a package, r the node's minimum fee rate, and for a component K let $\text{best}(K)$ be the maximum fee over its outcomes and $\text{best}_{\neg C}(K)$ the maximum over its outcomes of the fee excluding the candidates' own fees.

Definition 4.2 (Coverage)**.** *A package is* covered *when*

$$\sum_{K} \big(\text{best}(K) - \text{best}_{\neg C}(K)\big) \;\geq\; r \sum_{c \in C} s(c),$$

the sum over the components containing candidates.

The left side is the fee the package adds to the best outcome of each component it touches. The rule says that this added fee must pay for the package's own block space at the minimum rate. It is applied after the package has passed consensus checks, the block-level guards and the package rate pre-filter, and before script validation.

4.3 The coverage invariant

Theorem 4.1 (Coverage invariant)**.** *Under the marginal rule with rate r, the best outcome of every component pays at least r per chargeable byte of every row ever admitted into it, winners and losers alike.*

Proof. By induction on admissions. A component is created empty when the pool is latched. Consider an admission of candidates C into a component with best outcome fee b before admission. Every outcome that existed before admission remains an outcome afterwards: if the candidates are contested, the outcomes in which they lose are exactly the prior outcomes, since a just-admitted row has no descendants to remove; if a candidate is uncontested it joins every outcome and raises each by its fee. Hence $\text{best}_{\neg C} = b$ and the rule gives $\text{best} - b \geq r \sum_c s(c)$. Adding this to the inductive hypothesis $b \geq r \sum_{\text{prior}} s$ yields the claim. When an admission merges components, the hypotheses of the merged components add, and the same step applies. □

The invariant is the "all bytes" accounting that the design began from: the highest-fee outcome covers the bytes of every conflicting subgraph, including those of the stored instance that loses. The two formulations are equivalent, and the marginal form is the incremental way to maintain the other. A separate "bump" rate, a lower rate for the increment, breaks the invariant on its first use: at a bump rate of zero a lower-fee conflict is admitted whose bytes nothing in the best outcome covers. The design therefore uses one rate.

Lemma 4.1 (Own fee bound)**.** *The fee a package adds to the best outcome never exceeds its own fee.*

Proof. Let O^* be the best outcome with the candidates. Removing the candidates from O^* leaves a conflict-free, ancestry-closed set, since no non-candidate row descends from a just-admitted candidate. Its fee is $\text{fee}(O^*) - \text{fee}(C)$, so $\text{best}_{\neg C} \geq \text{fee}(O^*) - \text{fee}(C)$ and $\text{best} - \text{best}_{\neg C} \leq \text{fee}(C)$. □

Coverage therefore implies that the package's own fee meets the rate on its own chargeable size. The own-rate test remains in the chaser as a pre-filter, because it rejects without touching the pool, but it is not an independent rule. Coverage likewise implies bitcoind's absolute fee requirement for replacements, since a positive increment means the component's best absolute fee rose.

4.4 Block fit

The invariant credits the whole best outcome's fee to the component. If that outcome cannot be mined in one block, the credited fee is not one miner's prospect: whoever wins the next block takes a prefix, and the rest waits for a winner who may be anyone. For the network the bytes are still paid for eventually, so the invariant holds, but the fee-potential argument for keeping every contender does not. Two rules restore it.

Component cap, at coverage.

The chargeable size of a component, its members plus the closure of their unconfirmed uncontested ancestors, must not exceed a block's weight less its header, its largest transaction count and the configured coinbase weight, nor the block's sigop limit. Otherwise the candidate is refused with `pool_limit`. The coinbase weight is the miner's declaration of its own coinbase, and on a node that does not mine a guess at other miners'. Sigops need no reserve, since a coinbase paying to a script hash, witness program or taproot output counts none. One weight check covers both block size rules, because weight is at least four times size. Every outcome of every component therefore fits one block, and the best outcome's fee is one miner's prospect.

Uncontested candidates.

An uncontested candidate forms no component and is not capped. Its fee cancels in every comparison the rule makes, so it cannot be credited past a block boundary, and a chain of any depth is admitted. The template bounds the ancestry it will walk at once by a configured chunk depth, and takes a long chain level by level [10].

The uncontested ancestry is included because the contested closure excludes it, yet a miner must include it to mine any outcome. The closure is a marking walk up the parent rows that stops at boundaries and at rows with no pool parent, and its size is the sum of chargeable sizes over the rows reached.

4.5 Limits

Three limits bound the computation; none is an economic rule.

- *Members.* A component holds at most N rows, one per bit of the mask word in which outcomes are represented, where N is 64, 128, 256 or 512 as the available lane width allows. The passes do not change with N; the implementation measured here uses 64.
- *Outcomes.* The outcome count, the product of clique sizes, is capped by the node setting `maximum_outcomes`, defaulting to 1024. Evaluation is one pass over the members per outcome, so the work per admission is exponential in the number of independent contests: ten binary contests give 1024 outcomes and twenty give a million. Finding the best outcome is a maximum-weight conflict-free ancestry-closed subset problem, hard in general, so exact evaluation is enumeration and enumeration must be bounded.
- *Size.* The component cap above.

A package refused by any limit receives `pool_limit`, a database error code distinct from `insufficient_fee`. The node does not count it as peer misbehavior, since a peer cannot see the extent of the component this node holds. The honest cost of the limits is that a paid-for candidate landing in a component past a cap is refused; the defaults are a judgment about how much contest structure one node will reason about.

4.6 Context

Lock times are minimums and consensus flags change rarely, so a transaction validated under the pool context is confirmable in any later compatible context. Compatibility is settled in the validation step, and the index records only the result. A fork that can invalidate pooled transactions forces a single revalidation of the pool's members at the fork latch; members that

fail become absent and are purged with the other unconfirmables at prune. Context is stored once per header, where it never changes, and once for the presumed next block, under which pool validation runs; nothing about it is stored per row.

4.7 Alternatives considered

First seen.
The original bitcoind rule: the first transaction to spend a point is kept and every later conflict is dropped. It needs no fee reasoning and protects nothing but the first relayer's luck; a miner holding only the first-seen set forgoes every later, better-paid contender.

Cluster mempool.
The current bitcoind proposal [7], compared at length in Section 8. It resolves conflicts at admission by a fee-rate diagram test and keeps one survivor.

BIP125 replace-by-fee.
Rejected. Its rules, no new unconfirmed inputs, a cap on evicted descendants, replacement of the whole conflicting set, exist to make a resolution safe. Without resolution they have no object.

All-bytes accounting.
Equivalent to the marginal rule by Theorem 4.1; the marginal rule is its incremental form.

A separate bump rate.
Rejected; it breaks the invariant.

A stored dependency graph.
The 2021 "transaction pool graph" as a populated structure. Rejected in favor of derivation by vectorized passes, Section 5; the graph is still the object, but it is computed, not kept.

5 Algorithm

5.1 Labels

Each pool row holds one 32-bit *cell*: a label in the low 30 bits and a state in the top two, one of absent, present or boundary. A label is a row index. Boundary and absent cells read as *unlabeled*, a value greater than every row index, so a minimum over labels ignores them without a test. One fixed unlabeled cell past the last row, the sentinel, is the target of every parent reference to a transaction that never had a pool row, so that gathers over parents are unconditional.

5.2 Propagation

A present relevant row repeatedly replaces its label with the minimum over itself, its relevant parents, its children and the members of the cliques of its contested points, until no label changes. At the fixed point every row of one component carries the same label, the lowest row index in the component. Rows that are not relevant are never labeled.

Propagation is monotone: labels only decrease within an event, and a reader that observes a torn intermediate state is harmless because it runs the same iteration to its own fixed point.

5.3 Collection, relation and evaluation

Given a candidate's label, *collect* gathers the rows carrying it, failing above the member bound. *Relate* computes, for each member, its ancestor mask and the mask of each clique, failing if there are more cliques than the mask holds. *Outcomes* is the product of clique sizes; $outcome(i)$ decodes selection i by mixed radix into a member mask, clearing losers and their descendants through the ancestor masks. *Evaluate* computes, in one pass over the members per outcome, the outcome fee and the outcome fee excluding the candidates, by broadcasting each member's

fee and selecting on its mask bit, with no gather. The maxima are best and $\text{best}_{\neg C}$. *Covered* applies the rule.

5.4 Two forms of the same passes

Every pass is one of a few regular kernels over aligned arrays: gather, lane minimum, compare, select, masked sum, masked store. There is no pointer and no data-dependent branch inside a pass, and no ordering is produced. The passes have two forms.

Whole-column.
Each pass visits every row. This is the shape a GPU wants: one thread per row, a grid synchronization per pass, and a fixed-point loop on the device.

Frontier.
Each pass visits only the rows whose label changed in the previous pass plus their neighbors. With a maintained index (Section 6) a single admission changes the new row and whatever merges through it, so the rows touched are bounded by the component and its ancestry, independent of pool size. Collection disappears as a separate pass, since the rows reached are the component.

The CPU runs the frontier form; the GPU runs the column form; threads over column slices serve the passes that remain whole-column on the CPU, such as a prune rebuild or a fork revalidation. All three dispatch from the same kernel bodies.

6 The persistent index

6.1 Columns

The pool is a hashmap keyed by transaction link whose spine holds the validation context and time. Beside the spine are word-aligned no-map columns, one record per pool row:

column	bytes	content
`id0..id3`	4×8	witness-hash lanes for the compact block short-id scan
`totals`	16	fee (8), virtual size (4), sigops (4)
`edges`	8	first spends input (4), input count (4)

The `spends` cache keeps its single append-only column, one parent transaction link with the coinbase bit per pooled input, contiguous per transaction. Confirmation depends on it: confirming a block calls `get_pooled` for every pooled transaction in the block and fills its prevouts from these links with no point lookup, and the usual parent is confirmed outside the pool, so its link exists nowhere else on that path.

Everything the index changes after admission lives in value heads, the store's in-place primitive used by the strong and state indexes: a head of one 32-bit cell per key, with no body, written to its file only by explicit transfer at snapshot.

head	keyed by	cell holds
`cells`	pool row	state (2 bits) and label (30 bits)
`children`	pool row	first input spending the row, none if childless
`rows`	input	the row the input spends, none if not pooled
`siblings`	input	next input spending the same row, none at the end
`rivals`	input	next input spending the same point, a ring, none if sole

Bodies are staged and append only, so settled records are never rewritten; the lists and rings are threaded through existing cells at every admission, and labels change at every transition, which is why these values are heads and not columns. The owner of an input is not stored: the `edges` column is ordered by first input, so a binary search over it yields the row. Five heads of one value each keep every array a single 32-bit lane for the passes.

Heads are resident by construction, but the index heads are configured like the strong index rather than the hash heads: file-backed with sequential advice and no touch pass or wiring, so their pages are droppable. The frontier walk touches a few dozen cells per admission, scattered, and a fault on a cold cell costs no more than a probe would have; the whole-column passes that want the arrays warm run at prune or on the GPU, where they are staged. Growth is unbounded between prunes, about 22 MB per day at mainnet rates for the five heads, so compaction cadence, not the head, bounds the resident set.

6.2 Zero copy

The coverage query takes spans over the cells, spends and totals columns directly from the memory map, plus a short span for the candidates, which are not yet rows. Nothing is translated or copied per call, and the one million strong-table probes per admission of the first implementation, which classified every row as present or confirmed, are replaced by the state bits.

6.3 Events

The index is created empty with the pool head when the pool is latched; the pool has no rows at that moment, so there is no build step. It is persistent thereafter and maintained in place, like the chain's height and strong indexes, through five events.

Admission.
Write the row's cell as present with its own index and its spends cells, then propagate from the new row until the merge settles. Admission can merge components but never split one, so labels only decrease and a few local passes reach the fixed point.

Block in.
Each confirmed transaction's cell becomes boundary. A boundary can split its component, which a minimum label cannot shrink incrementally, so the component is relabeled; it is bounded at N members. The confirmed transaction's conflicting spenders are now unconfirmable and become absent.

Block out.
Each reorganized transaction's cell returns to present, the rivals it had excluded are revived where confirmable with their descendants, and each affected row is relabeled into its component, a merge-only step. The conflicts listed in the block's prevout cache, which cover transactions that were never pooled, are revived the same way. A reorganized transaction that never had a row, having confirmed before the latch, is admitted as usual; its pooled children hold its link in the `links` column, so an equality scan over that column finds the inputs whose `rows` cell must become the new row. The scan is a vector pass and needs no index walk.

Fork latch.
A fork that can invalidate pooled transactions revalidates the members once; failures become absent.

Prune.
Compaction copies retained rows into a fresh pool with their links, then rebuilds the five heads from the compacted pool: parent rows by link, lists from the parent rows, rings from the point multimap, and labels by the column passes. Retained rows are all confirmable, so all are present.

Cells are assigned atomically. The transaction chaser and the confirmation chaser each run on their own strand and may touch the index concurrently; monotone labels and the fixed-point iteration make this safe without a lock.

7 Computation

7.1 The shared pattern

Libbitcoin's hash and curve code is written once over a word type `Word` with a namespace of functional primitives, `f::`, that has a `constexpr` integral overload and one extended overload per instruction set (SSE4.1, AVX2, AVX-512, NEON), plus a mock for platforms without. Dispatch is at compile time on `have_*` predicates and at run time on CPU features, and `std::is_constant_evaluated` selects the integral path in constant expressions. The CUDA path compiles the same template bodies at `Word = uint64_t` with `clang -cuda-device-only` into PTX, which is deflated into the library and loaded through the driver API, with columns staged at 256-byte alignment.

The pool algorithm follows this pattern exactly. Two primitives were added to `f::`: a lane minimum and a 32-bit gather. On AVX-512 the unsigned 32- and 64-bit minimum and the 32-bit gather are F instructions and the byte and word minimums are BW, so the whole algorithm sits within the F/BW/VL target the library requires, and no population count instruction is needed because outcome fees are summed by broadcast and select rather than by counting bits. Each pass takes full words at each available width, widest first, and finishes one element at a time, as the merkle hashing does, so no lane is ever left empty: seven outcomes on AVX-512 are one 256-bit, one 128-bit and one scalar step. The SSE4.1 and NEON tiers provide two 64-bit or four 32-bit lanes, so a two-outcome component is still one full vector step.

7.2 Table layout and scans

The compact block short-id correlation already scans the `id` columns with the same lane widths, so the pool has one computational model for everything it does. Columns are word-aligned so that a span over the memory map is a valid lane source, and adding or moving a column leaves the existing scan untouched. The main cost in all of this is paging, which is why the state bits replace probes and why nothing is copied.

8 Comparison with the cluster mempool

8.1 Cluster mempool

The cluster mempool proposal [7], now implemented in bitcoind, partitions the mempool into clusters, the connected components of the unconfirmed spend graph, capped at 64 transactions and at a cluster weight. Every cluster keeps a linearization, a topologically valid ordering optimized for the fee-rate diagram, maintained incrementally in memory by the TxGraph structure. Mining takes the best chunks across clusters, eviction drops the worst, and a replacement is accepted when the diagram of the affected clusters after the change is nowhere below the diagram before, with the replacement also paying for its own relay bandwidth. Conflicts never coexist: one contender survives admission and the rest are discarded.

8.2 Three fundamental differences

Conflicts coexist.
bitcoind's mempool is conflict-free by definition and admission is the conflict resolution. Ours admits every contender that pays for its block space and never resolves; resolution is

deferred to the miner at template time. bitcoind decides for the miner at admission; we hand the miner the whole paid-for set.

An index over an archive, not a store.
bitcoind's mempool is a separate in-memory structure holding transaction data and a stored graph, rebuilt from a dump at start, evicted under RAM pressure and bounded in size. Ours is columns over transactions already archived, persistent and maintained through the events the archive sees, with no graph stored and no rebuild. Once archived, a transaction remains in place; a block that later includes it associates it where it lies. Admission to the pool is admission to the store, so the denial-of-service question is purely whether bytes written are paid for, and the invariant answers it with no eviction policy.

Derived by column passes, not maintained per cluster.
bitcoind keeps a linearization per cluster and updates it with scalar graph algorithms that approximate the best ordering. We keep one label per row, recover components by vector propagation, and evaluate outcomes exactly under a cap. bitcoind answers "in what order should these be mined"; we answer "which paid-for sets are mutually confirmable".

8.3 Consequences

- bitcoind needs an incentive-compatibility test for replacement; we need none. The diagram check asks whether, for every amount of block space, a miner building from the new mempool collects at least as much as from the old; it exists because bitcoind's acceptance is itself the resolution, a judgment made on the miner's behalf. We make no judgment, so there is nothing for a diagram to test. Incentive compatibility holds by construction: the party with the incentive makes the choice.
- bitcoind needs eviction and a mempool size; we need prune of the unconfirmable after confirmation.
- bitcoind's cluster is every connected transaction; our component is only the contested closure, so uncontested chains have no structure and no bound beyond block fit.
- bitcoind's limits are per cluster in count and weight; ours are per component in members, outcomes and chargeable size.
- Both price block space the same way, adjusted size at one rate, and both bound what is credited to what one block can mine. Both treat confirmed transactions as boundaries, reinsert on reorganization, and bound a component by a word-sized mask, 64 in both as implemented.
- bitcoind never admits a lower-fee conflict. We admit one that pays for its own bytes, because the best outcome still covers everything.

Where we are weaker in kind: exact enumeration is exponential in independent contests, so the outcome cap refuses structure bitcoind would linearize approximately. Where we are stronger in kind: no fee-paying contender is ever discarded, and the structure is recoverable from the archive by anyone, which an in-memory mempool is not.

8.4 Computationally

bitcoind's TxGraph is a heap of entries linked by pointers, and linearization is a search, spanning-forest and candidate-set algorithms that branch on data, backtrack and improve an ordering incrementally. Work per event is small, latency-oriented and irregular, so it runs on one core in scalar code. Its subset bitsets are the one place it resembles ours. Our inputs are aligned arrays read in place, and every operation is a regular kernel with no pointer and no branch inside a pass. Propagation parallelizes across rows; outcome evaluation is one independent thread per outcome; block in and block out are masked stores and a bounded relabel. The constant per event is higher where the work is trivial, and contests still come from hash probes that no lane width helps; but cost does not grow with graph complexity, only with rows and passes, and the

whole computation moves to a GPU without redesign because nothing in it was ever a graph.

9 Cost estimates

All figures in this section are estimates from bandwidth, instruction throughput and algorithm structure. None is a measurement. They are for a pool of one million rows resident in memory on the reference machine: a 64-core Zen 5 workstation with full-width AVX-512 and 512 GB of DDR5-6400, and a Blackwell workstation GPU with 96 GB. One million rows is about 40 MB of columns (cells 4, totals 16, edges 8, spends about 9 at 2.3 inputs per transaction), inside the CPU's 256 MB L3 and trivially resident on the GPU. Script validation of a two-input transaction on this machine is on the order of 0.1 ms, which places every figure below in context.

9.1 One whole-column pass

path	per pass	basis
scalar, one core	∼10 ms	2.3M random loads at 2–3 ns plus the scatter-min
AVX-512, one core	∼5 ms	gathers are latency bound, so lanes buy about 2×
AVX-512, 64 cores	∼0.2 ms	∼80 μs of work; fork, join and imbalance are the rest
GPU, resident	∼20 μs	30 MB at ∼1.5 TB/s plus a grid synchronization

Lane width pays most where a pass streams rather than gathers: a collect over the cell column is about 0.3 ms scalar and 0.05 ms at sixteen lanes. Such passes are a minority, so the gather figure sets the pass cost.

9.2 Three levels of complexity

Low.
An uncontested candidate whose parents are confirmed or uncontested. No pass runs; the cost is the point probes and the cell writes.

Medium.
One conflict; a component of about ten members with two or three cliques, depth three, about thirty outcomes.

High.
A component at the member bound, about ten cliques, the outcome cap, chains about 25 deep.

Table 1 gives the per-admission cost of the whole-column form on each path, and of the frontier form, which touches only the component and its ancestry and so is independent of pool size and lane width. "Passes" counts propagation to the fixed point, collection and ancestry marking.

The GPU figures assume the fixed-point loop runs on the device with a grid synchronization per pass; a host launch per pass with a flag read back adds about 10 μs per pass and roughly doubles medium and high. The 64-core figures are dominated by thread fork and join; a thread pool kept warm for the passes brings medium under 1 ms.

9.3 bitcoind at the same levels

Table 2 estimates the cluster mempool's acceptance cost for the same three shapes, excluding script validation in both designs. The estimates follow the algorithm's structure: TxGraph

level	passes	scalar	AVX-512	64 cores	GPU	frontier
low	0	5 μs	5 μs	5 μs	5 μs	5 μs
medium	$\sim$7	70 ms	35 ms	1.5 ms	0.2 ms	20–50 μs
high	60–90	0.8 s	0.4 s	15 ms	2 ms	0.1–0.3 ms

Table 1: Estimated cost of admitting one transaction into a one-million row pool. The first four cost columns are the whole-column form on one scalar core, one AVX-512 core, 64 AVX-512 cores and the resident GPU; the last is the frontier form on any single core.

insertion and cluster merge are pointer operations on small objects; a merged cluster is relinearized by the spanning-forest algorithm under an iteration budget; a replacement additionally linearizes the clusters before and after the change and compares their diagrams.

level	work	estimate
low	insert into TxGraph, trivial cluster, no relinearization	10–20 μs
medium	merge into a $\sim$10-tx cluster, relinearize; for a replacement, linearize before and after and compare diagrams	0.1–0.5 ms
high	64-tx cluster relinearization to the budget; replacement doubles it	1–5 ms

Table 2: Estimated acceptance cost in bitcoind's cluster mempool at the same three levels, one core, scalar.

The comparison that matters is frontier against bitcoind on one core, since both are the single-admission path a relay node runs: the two are within a small factor at every level, with ours lower at medium and high because it enumerates a bounded component rather than searching an ordering, and bitcoind lower in constant at low. The GPU and 64-core columns are not a per-admission advantage at one million rows; they become one for the batched work below.

9.4 Larger candidate sets

Three kinds of larger input arise. A *package* of k dependent candidates is one admission: the candidates are labeled together, the components they touch are evaluated once, and the cost is that of the single-candidate level of the merged component plus k point probes. A *burst* of n unrelated transactions, as relay delivers, is n independent admissions. A *compact block fill* admits the block's missing transactions, typically a few thousand, under proof of work and therefore without coverage; its row in the table is the cost of maintaining the index for them, an admission write plus propagation per transaction, with no evaluation.

Batching is where the column form earns its place. A burst or a fill of thousands of transactions is one propagation of a few passes, not thousands, so the per-transaction cost falls to the write and the probes, and the GPU figure is a handful of launches regardless of n. bitcoind's cost is linear in n at its per-transaction constant, with relinearizations where the burst lands in clusters.

10 Open items

Template construction over outcomes belongs to the version 5 metadata graph, for which this tx pool is the persistence. What that implies is already fixed by the index. A template is a choice

input	frontier, 1 core	64 cores	GPU batched	bitcoind, 1 core
package, $k = 5$, medium	30–60 μs	—	—	0.2–0.6 ms
package, $k = 25$, high	0.2–0.4 ms	—	—	2–6 ms
burst, $n = 1000$, mostly low	5–10 ms	0.2–0.5 ms	0.1–0.2 ms	10–20 ms
burst, $n = 1000$, 10% medium	10–20 ms	0.5–1 ms	0.2–0.4 ms	20–70 ms
compact fill, $n = 3000$	15–30 ms	0.5–1 ms	0.1–0.3 ms	30–60 ms

Table 3: Estimated cost for larger candidate sets. Frontier and bitcoind are per-admission costs summed. The 64-core and GPU columns batch all admissions into one set of whole-column passes: independent rows are written at once and a single fixed-point propagation settles every merge. A package is one admission and does not batch.

of one outcome per component and a packing of the chosen outcomes, with their uncontested ancestry, into the block's weight and sigop limits. The outcome enumeration gives the choices and their fees, the ancestry walk gives their chargeable size, and the labels partition the pool so that the choices are independent across components. Template construction is then a packing over components, the only place an ordering is ever produced, and it is the whole-column form of the passes, run once per template rather than once per admission, that makes it a kernel: every component is evaluated at once, on the lanes or on the GPU, and the miner's criterion for choosing among outcomes is a parameter of the kernel, not of the pool. The version 5 graph adds the objects that express that choice; it does not add state, because the pool already holds all of it.

Fee estimation is the template run forward. The estimate for a target of k blocks is the lowest rate at which a new transaction would be mined by this node in the k-th block from now, block zero being the template it would build at this moment. That is a query over the pool and not a statistic over history, so it keeps no accumulator, no decay and no state of its own. The pool is what makes the definition computable. It removes nothing by age, so everything the node would mine is in it; the outcome choice resolves each conflict to the set a miner would take; and the chunk rate of the packing [10] already prices a child with its parents. The projection packs K blocks in sequence, each from what the block before left, and records each block's takes as chunk rate and chargeable size. A new transaction with no pooled parent, of size s and rate r, changes no other chunk, so it is in block k when it was not taken earlier and the space that block gives to chunks rated above r, plus s, fits the block. K runs until the pool is exhausted, and a target beyond that depth is answered by the node's minimum rate.

A projection of the present pool assumes nothing else arrives, which is true only of block zero. The pool measures arrivals itself. Each row keeps the context it was validated in, and the height of that context is that of the next block when it arrived, so the chargeable bytes that entered over the last N blocks at rate r or above give an arrival rate $\lambda(r)$ per block, and the projection for block k reserves $k\lambda(r)$ of competing space above r. Arrivals are counted in blocks because the estimate is asked in blocks; the row holds no arrival time, its timestamp being that of the context, which validation requires. The estimation modes choose N and a quantile of the arrival rate, a short window for an economical estimate and a long window at a high quantile for a conservative one. Rows that confirmed are arrivals too, and they remain as boundaries until prune, so the window reaches back to the last prune, which the node runs once in a run, when it first becomes current. An estimate formed sooner than N blocks after that has a shorter window, and what preceded a restart is stale in any case. The same heights give a check on the model: a transaction that the projection places in an early block and that has waited many blocks says that miners are not building what this node would, and by how much.

The projection reads the columns the template reads and stores nothing. It runs where the template runs, on the strand that orders every pool write, between writes and without a lock,

and its result is kept against a count of those writes, so an estimate requested of an unchanged pool is a lookup. A node that mines already packs block zero at every change, which is the first target; a node that does not mine pays for a projection only when one is asked for, and such requests are far rarer than admissions. The cost is K packings, and it falls with the same change that would speed the template, an ordering of chunks by rate that is maintained across takes rather than rebuilt for each round. The implementation does not yet build the projection, and answers a fee request with no estimate.

Relay is the same walk run against a peer's view of the pool. Each channel holds marks, the rows the peer is known to hold, and a rate. An accepted transaction's unmarked unconfirmed ancestry is walked, accumulating fee and chargeable size until the aggregate clears the peer's rate within block scope, and coverage is evaluated with the marked rows as the table and the unmarked ones as candidates. The result is a package to announce or a deferral, the take-or-drop decision of admission made on the peer's behalf, from the same kernels, with marks as one more head keyed by pool row. None of this modifies the protocol. The package is formed when a peer needs it and sent as a set of ordinary transactions, and what the peer does with them is its own affair. On receipt the node reconciles an incoming set against its pool as one admission. A set with a gap, a child whose parent is neither pooled nor present, or with an extra, a transaction that neither clears the rate nor pays for one that does, is the sender's violation of the rule, and the offending peer is dropped. That is why an orphan is never received: one arriving is a gap, and the peer that sent it is gone. The node does not implement bitcoind's ancestor package relay proposal [9], still a draft; the expectation is that the network evolves toward narrowcast, which asks nothing of it.

Rates change, ours and theirs, and the index absorbs both without retroaction. The node's minimum rate is a setting, fixed for a run and announced to each peer by fee filter. Raising it rejects new candidates and nothing else: a row admitted under the old rate was a valid candidate when admitted and remains one, the pool being an index over an append-only archive with confirmation as its only exit, and it remains relayable to any peer whose rate it clears, which may be lower than the node's own. Lowering it admits more and recovers nothing, since a rejected transaction was never stored; what is recovered comes from peers that re-sweep on the new filter, as this node does. A peer's rate is held per slot and changes by its fee filter. A rise leaves the marks alone, since what the peer holds it holds, and only judges the next package against the new rate. A fall is the one event that creates work without a transaction arriving: unmarked rows that were deferred under the old rate may clear the new one, so the drop triggers one sweep of the peer's pool, the same marks-and-rate pass the next transaction would run, over the whole column rather than a frontier. Fee filter changes are infrequent by convention and rate-limited by the sender, so the sweep is bounded by the pool size times the filter rate, and like every other pass here it is one kernel over the columns.

The relay path can also stop deserializing. A transaction view is the wire bytes with their offsets, and it computes txid and wtxid from those bytes, so a received transaction becomes a view by one copy out of the channel's read buffer, and every relaying channel writes that same buffer behind a heading whose checksum is the cached hash. The bytes received are the bytes sent, and no object is built or serialized in between; the pool index already holds the wtxid for every row, so a transaction relayed from the pool never hashes either. Validation runs on the view directly: its check, accept and connect are the object's, and the contextual checks that the object path reads from input metadata, maturity, relative locks and spends, are archive reads keyed by the transaction's points, the same split the block view already makes. What the view lacks for this is its own population, the inputs and prevout buffer the block view owns on behalf of its transactions, and the population guard that goes with it; a package admitted as one set also needs internal population, since a child's prevouts may be the unarchived parent's outputs. The implementation uses whole transaction objects, which are fast and simpler; the view path is a downstream optimization of the same design, not a change to it.

References

[1] E. Voskuil. *Libbitcoin Version 4 Node*, version 8, 6 November 2021. Published on the project site: `https://libbitcoin.org/architecture/node/libbitcoin-v4-node-design.pdf`. Cited passages: Candidate Chain (p. 2), Validated TX cache and pool DoS guard (pp. 13–14), optimization goals (p. 16), transaction check, accept and connect (pp. 21–24).

[2] E. Voskuil. *Libbitcoin Version 4 Relational Database*, version 3, 10 November 2021. `https://libbitcoin.org/architecture/database/libbitcoin-v4-relational-database.pdf`.

[3] libbitcoin-node wiki, *12 — Periphery Chasers*, §4 (`chaser_transaction`), 2024: "awaiting a mempool design"; "a partial DAG ordered by dependencies, with insertion emitting per-tx events".

[4] libbitcoin.org, *Architecture: Node, 11 — Transaction Chaser*. `https://libbitcoin.org/architecture/node/transaction-chaser`

[5] libbitcoin.org, *Architecture: Database, 03 — Tables*. `https://libbitcoin.org/architecture/database/tables`

[6] libbitcoin.org, *Architecture: Database, 05 — Queries*, §6 Pool. `https://libbitcoin.org/architecture/database/queries`

[7] S. Daftuar and P. Wuille, *Cluster mempool*, design proposal and bitcoind implementation, 2023–2025; P. Wuille, *Spanning-forest cluster linearization*.

[8] E. Voskuil and P. Mienk, *Narrowcast Transaction Relay: Adaptive Packaging for Bitcoin Transaction Propagation*, August 2022, with a note added in proof, August 2026.

[9] G. Zhao, *BIP 331: Ancestor Package Relay*, 2022, status Draft.

[10] E. Voskuil, *Tidal Wave: Objective, Scalable Block Template Construction over a Semi-Persistent Bitcoin Transaction Pool*, 12 November 2026. Companion to this paper.

[11] D. Harding and P. Todd, *BIP 125: Opt-in Full Replace-by-Fee Signaling*, 2015.

[12] E. Lombrozo, J. Lau and P. Wuille, *BIP 141: Segregated Witness (Consensus layer)*, 2015: block weight and sigop cost.